Mount Kenya University

Student Name: Faith Wanja Gichure

Admission Number: BSCCS/2024/31978

Unit Code: BIT4133

Unit Name: Natural Language Processing with Deep Learning

NLP Extra Work

NLP Project Title: SMS Spam Detection System

Technologies Used: Python, NLTK, Scikit-learn, Google Colab, Pandas, Matplotlib

GitHub Link: https://github.com/faith-dev122/BIT4133-NLP-Project

Table of Contents

EXTRA WORK SECTION 1 WEEK I & 2

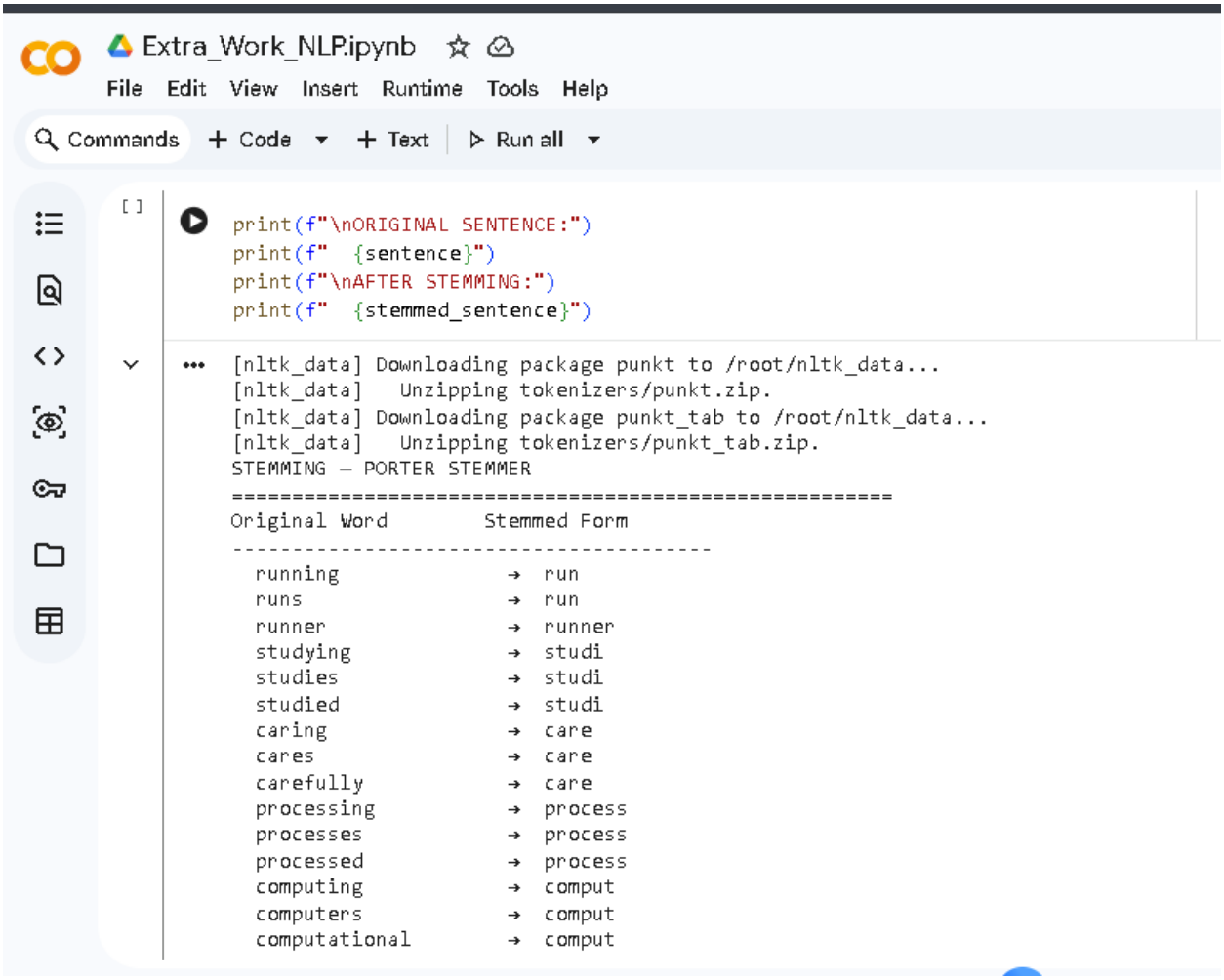


Fig 1- Stemming Output (Porter Stemmer)

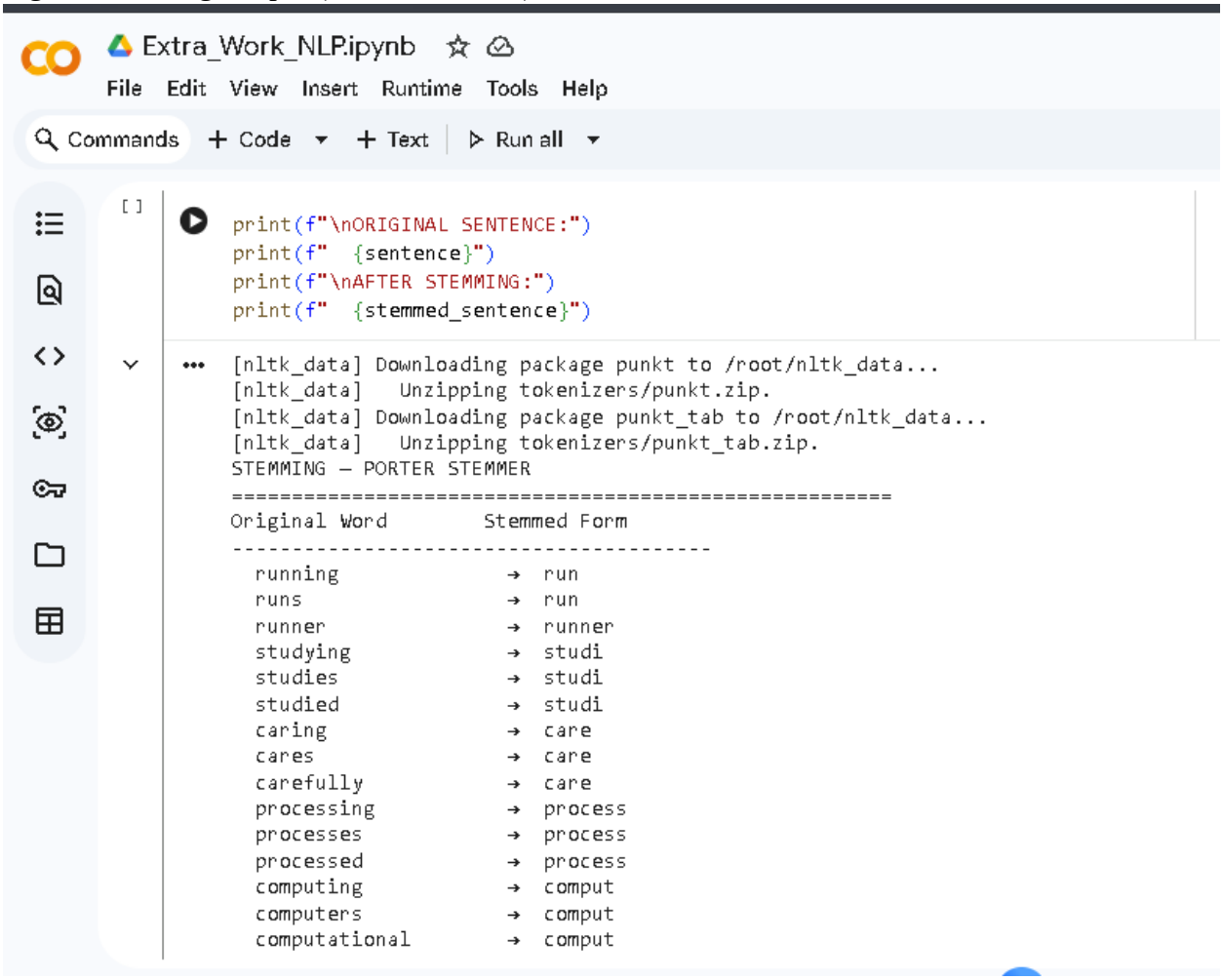
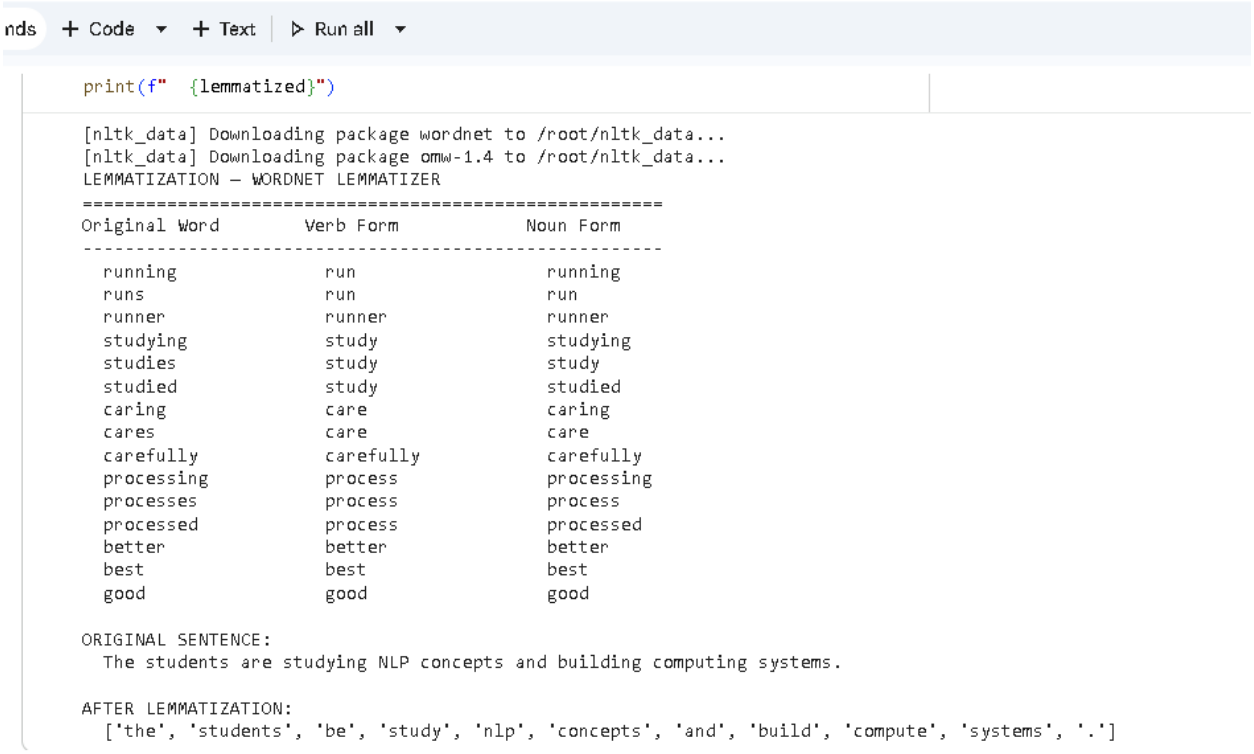
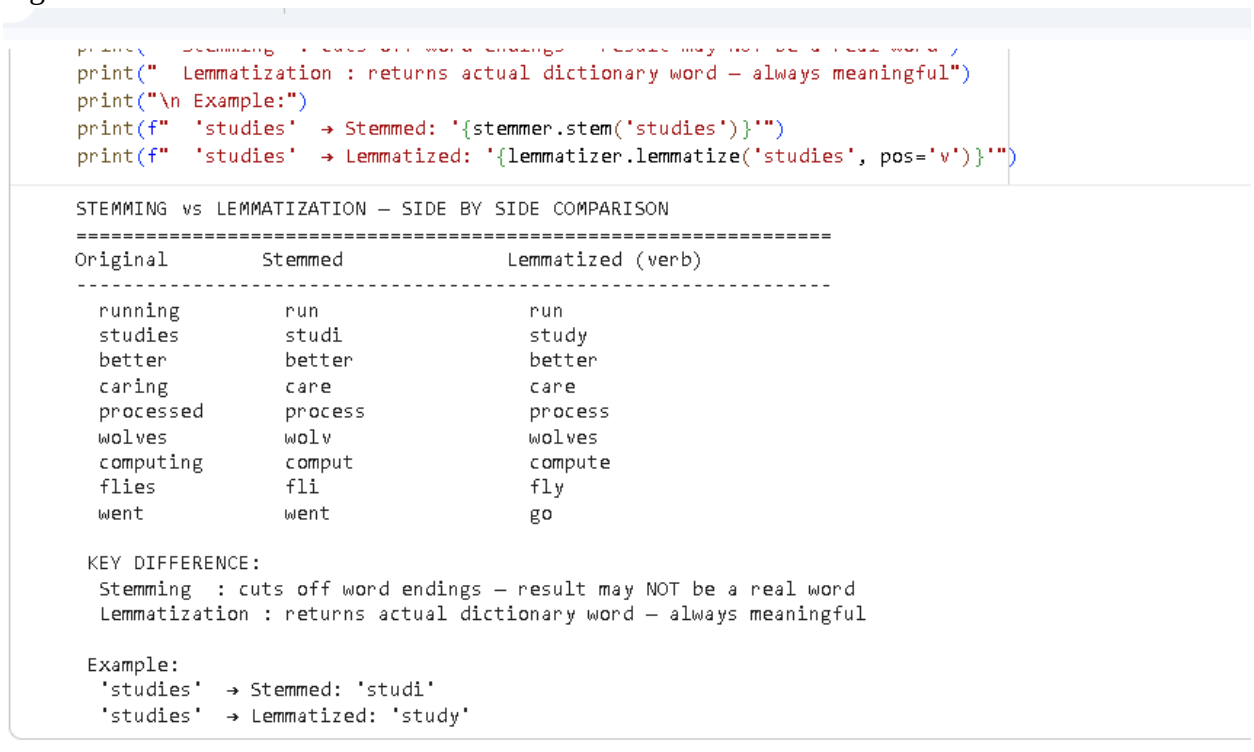


Fig 2- Stemming Output (Snowball Stemmer)



```
nds  + Code  + Text  ▷ Run all

print(f"  {lemmatized}")

[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
LEMMATIZATION — WORDNET LEMMATIZER
======================================================
Original Word        Verb Form            Noun Form
------------------------------------------------------
  running              run                  running
  runs                 run                  run
  runner               runner               runner
  studying             study                studying
  studies              study                study
  studied              study                studied
  caring               care                 caring
  cares                care                 care
  carefully            carefully            carefully
  processing           process              processing
  processes            process              process
  processed            process              processed
  better               better               better
  best                 best                 best
  good                 good                 good

ORIGINAL SENTENCE:
  The students are studying NLP concepts and building computing systems.

AFTER LEMMATIZATION:
  ['the', 'students', 'be', 'study', 'nlp', 'concepts', 'and', 'build', 'compute', 'systems', '.']
```

Fig 3- Lemmatization



```
print("  Lemmatization : returns actual dictionary word — always meaningful")
print("\n Example:")
print(f"  'studies'  → Stemmed: '{stemmer.stem('studies')}'")
print(f"  'studies'  → Lemmatized: '{lemmatizer.lemmatize('studies', pos='v')}'")

STEMMING vs LEMMATIZATION — SIDE BY SIDE COMPARISON
================================================================
Original        Stemmed              Lemmatized (verb)
----------------------------------------------------------------
  running         run                  run
  studies         studi                study
  better          better               better
  caring          care                 care
  processed       process              process
  wolves          wolv                 wolves
  computing       comput               compute
  flies           fli                  fly
  went            went                 go

 KEY DIFFERENCE:
  Stemming  : cuts off word endings — result may NOT be a real word
  Lemmatization : returns actual dictionary word — always meaningful

 Example:
  'studies'  → Stemmed: 'studi'
  'studies'  → Lemmatized: 'study'
```

Fig 4- Stemming vs Lemmatization comparison

SUMMARY

Stemming in Fig 1 and Fig 2 both removes suffixes from words to form a stem, for example the word running is reduced to the word run by both the Porter and Snowball stemming algorithms. Lemmatization (using the WordNet Lemmatizer) returns appropriate dictionary forms such as 'studies' to 'study', and ‘better' to ‘good'. (See Fig 3) The two techniques are compared in Fig 4. The difference is that stemming can generate non-real words, such as 'studi' while lemmatization always generates a real word. Lemmatizing is more accurate but slower, stemming is faster but less accurate. Both of them are crucial steps in our SMS Spam Classifier Pipeline that are performed prior to the processing.

EXTRAWORK SECTION 2 WEEK 3 & 2

```
        print(f"  {rank:<6} {word:<20} {count:<8} {bar}")

STEP 1 — TOKENIZATION
Total raw tokens: 90
After cleaning   : 60 meaningful tokens

WORD FREQUENCY COUNTER — TOP 15 WORDS
==============================================
Rank   Word                 Count    Bar
------------------------------------------------------
  1      language             6        ██████
  2      nlp                  4        ████
  3      natural              3        ███
  4      processing           3        ███
  5      understand           2        ██
  6      systems              2        ██
  7      fascinating          1        █
  8      branch               1        █
  9      artificial           1        █
  10     intelligence         1        █
  11     allows               1        █
  12     machines             1        █
  13     read                 1        █
  14     generate             1        █
  15     human                1        █
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Fig 1- Basic word frequency counter

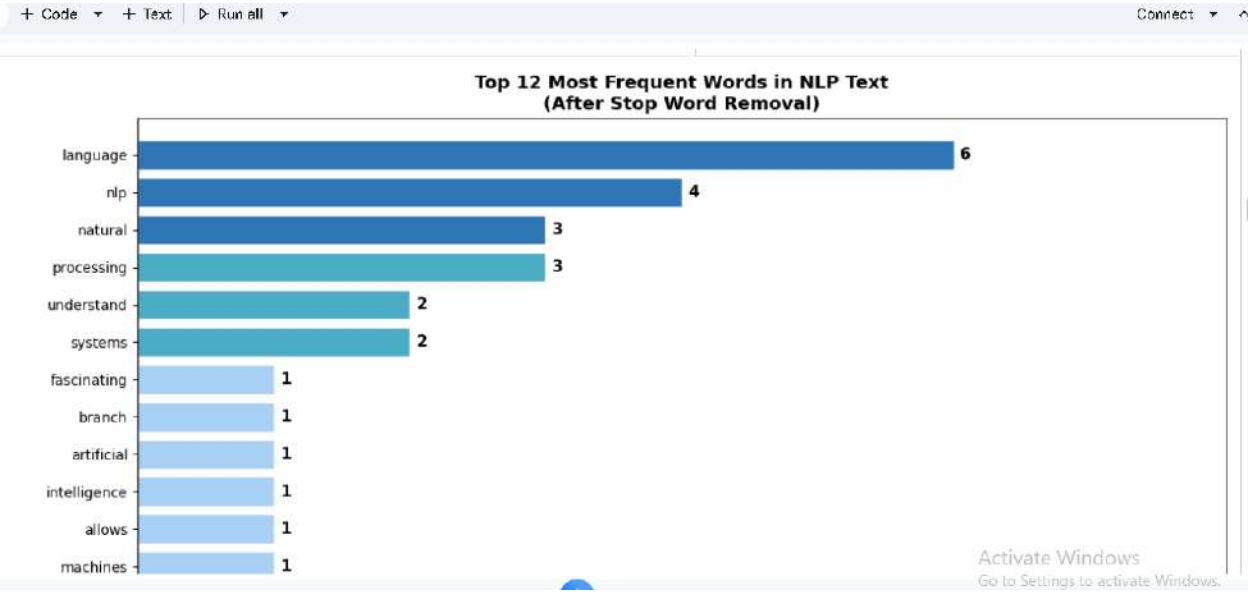


Fig 2- Word frequency bar chart

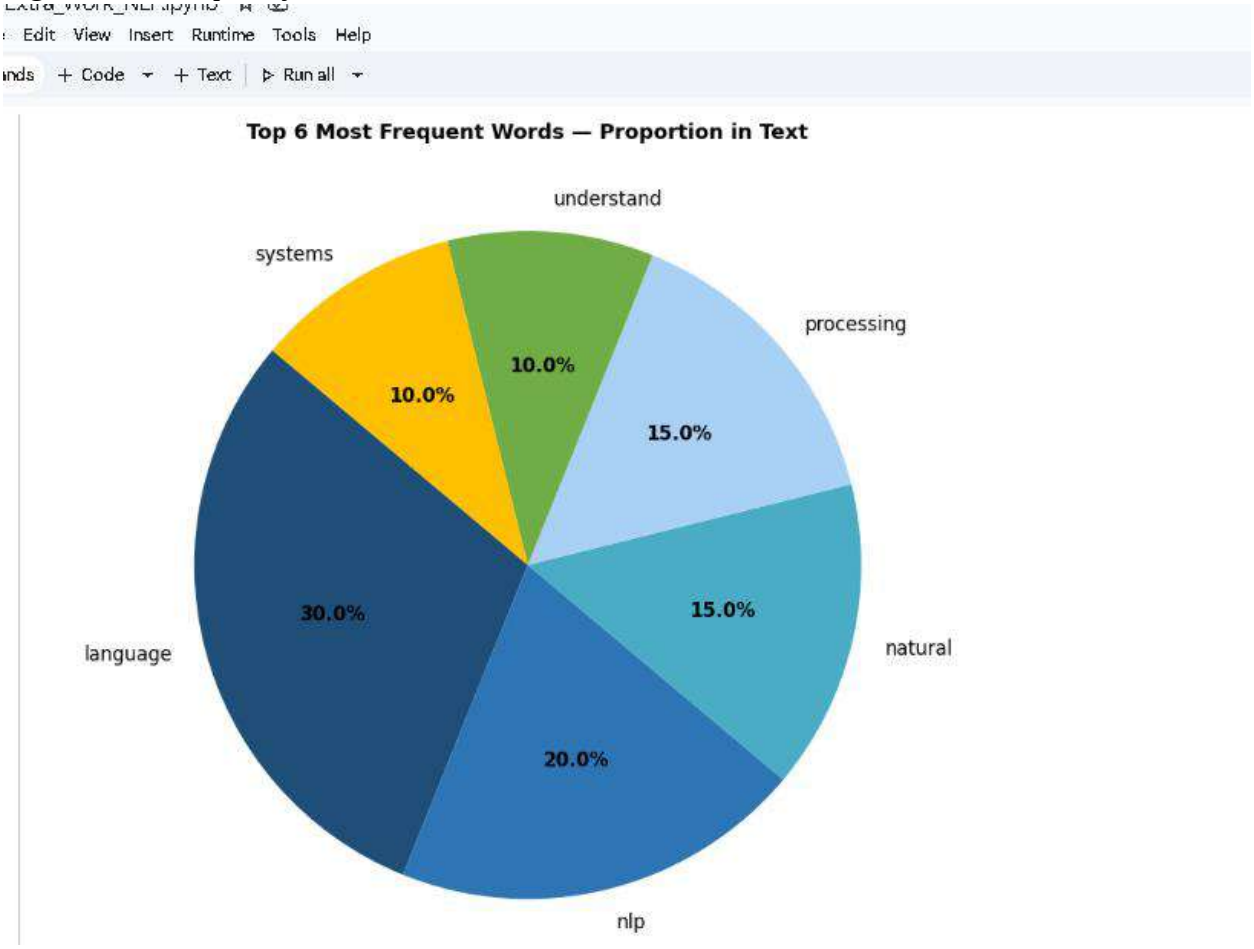
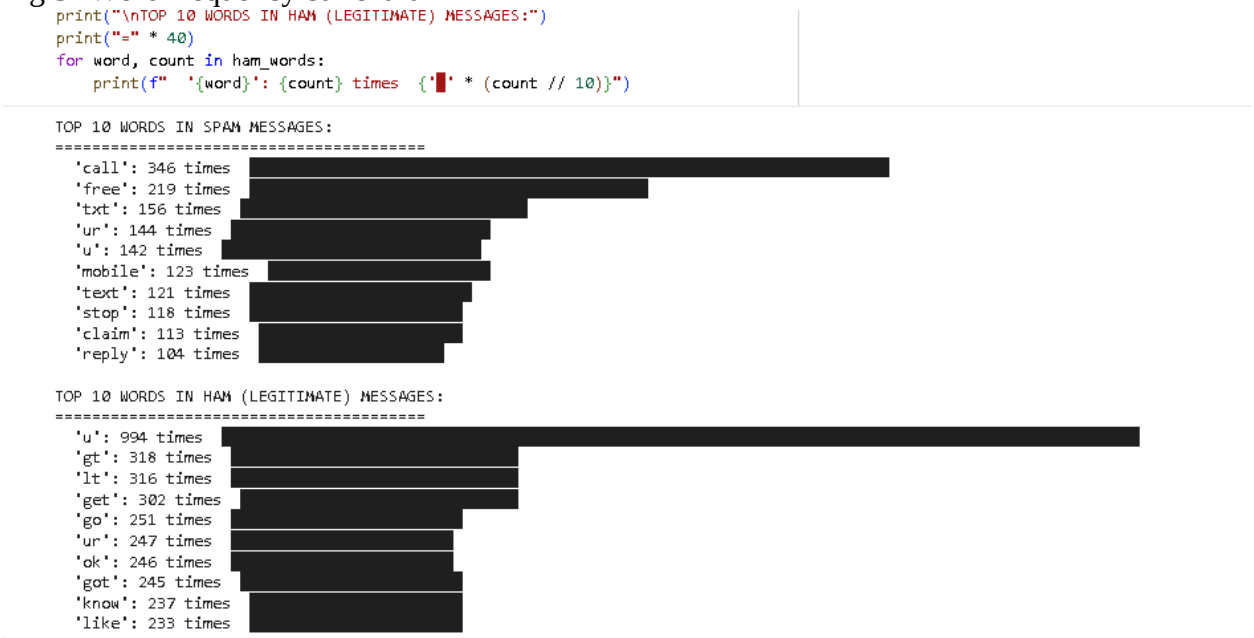


Fig 3- Word frequency bar chart



```
print("\nTOP 10 WORDS IN HAM (LEGITIMATE) MESSAGES:")
print("=" * 40)
for word, count in ham_words:
    print(f"  '{word}': {count} times  {'' * (count // 10)}")
```

```
TOP 10 WORDS IN SPAM MESSAGES:
========================================
  'call': 346 times
  'free': 219 times
  'txt': 156 times
  'ur': 144 times
  'u': 142 times
  'mobile': 123 times
  'text': 121 times
  'stop': 118 times
  'claim': 113 times
  'reply': 104 times

TOP 10 WORDS IN HAM (LEGITIMATE) MESSAGES:
========================================
  'u': 994 times
  'gt': 318 times
  'lt': 316 times
  'get': 302 times
  'go': 251 times
  'ur': 247 times
  'ok': 246 times
  'got': 245 times
  'know': 237 times
  'like': 233 times
```

Fig 4-Comparing spam vs ham word frequencies

SUMMARY

After tokenization and stop word removal, the most frequent words in the paragraph of the NLP text given in Fig 1 are highly concentrated on the central topics of the text, namely ‘nlp’, ‘language’, ‘processing’, and ‘learning’. The top 12 words are represented graphically in a horizontal bar chart (Fig 2) and as a pie chart (Fig 3) representing the proportions. With our own SMS Spam dataset, the word frequency analysis is shown in Fig 4 where words such as ‘free’, ‘call’ and ‘txt' are frequently used in spam messages while conversational words are more often used in ham messages. The basis of TF-IDF is word frequency analysis that is used in our spam classifier.

EXTRAWORK SECTION 3 WEEK 4

```
SYNTACTICALLY CORRECT SENTENCES — POS Tag Analysis:
------------------------------------------------------------

Sentence : The student studies Natural Language Processing.
Structure: [('The', 'DT'), ('student', 'NN'), ('studies', 'NNS'), ('Natural', 'NNP'), ('Language', 'NNP'), ('Processing', 'NNP'), ('.', '.')]

Sentence : Python is a powerful programming language for NLP.
Structure: [('Python', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('powerful', 'JJ'), ('programming', 'NN'), ('language', 'NN'), ('for', 'IN'), ('NLP', 'NNP'), ('.', '.')]

Sentence : The quick brown fox jumps over the lazy dog.
Structure: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('.', '.')]

Sentence : Nairobi is the capital city of Kenya.
Structure: [('Nairobi', 'NNP'), ('is', 'VBZ'), ('the', 'DT'), ('capital', 'NN'), ('city', 'NN'), ('of', 'IN'), ('Kenya', 'NNP'), ('.', '.')]


SYNTACTICALLY INCORRECT SENTENCES:
------------------------------------------------------------

Sentence : Student the NLP studies.
Structure: [('Student', 'NN'), ('the', 'DT'), ('NLP', 'NNP'), ('studies', 'NNS'), ('.', '.')]
  ⚠ This sentence has incorrect grammar/word order

Sentence : Is Python powerful a language.
Structure: [('Is', 'VBZ'), ('Python', 'NNP'), ('powerful', 'JJ'), ('a', 'DT'), ('language', 'NN'), ('.', '.')]
  ⚠ This sentence has incorrect grammar/word order

Sentence : Fox the lazy jumps brown over.
Structure: [('Fox', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('jumps', 'NNS'), ('brown', 'VBN'), ('over', 'IN'), ('.', '.')]
  ⚠ This sentence has incorrect grammar/word order
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```


Fig 1- Syntax Sentence Structure Analysis

```
print("KEY INSIGHT: This is why NLP is hard.")
print("The word 'bank' has", len(wordnet.synsets('bank')), "different meanings.")
print("Computers must use CONTEXT to determine which meaning is correct.")
```

```
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
NLP COMPONENT 2: SEMANTICS
============================================================
Semantics = the study of meaning in language

WORD AMBIGUITY — One word, multiple meanings:
------------------------------------------------------------

Word: 'bank'
Number of different meanings: 18
  Meaning 1: sloping land (especially the slope beside a body of water)
  Example : they pulled the canoe up on the bank
  Meaning 2: a financial institution that accepts deposits and channels the money into lending activities
  Example : he cashed a check at the bank
  Meaning 3: a long ridge or pile
  Example : a huge bank of earth

Word: 'bat'
Number of different meanings: 10
  Meaning 1: nocturnal mouselike mammal with forelimbs modified to form membranous wings and anatomical adaptations for echolocation by which they navigate
  Example : No example
  Meaning 2: (baseball) a turn trying to get a hit
  Example : he was at bat when it happened
  Meaning 3: a small racket with a long handle used for playing squash
  Example : No example
```

Fig 2- Semantics dealing with the meaning of words and sentences

```
print("  Modern AI (ChatGPT) uses pragmatic understanding")
print("  to respond to what you MEAN, not just what you SAY.")
```

```
NLP COMPONENT 3: PRAGMATICS
============================================================
Pragmatics = understanding intended meaning from context

Example 1:
  Sentence      : "Can you pass the salt?"
  Context       : Said at a dinner table
  Literal meaning: Asking whether you HAVE THE ABILITY to pass salt
  Actual meaning : A polite REQUEST to pass the salt

Example 2:
  Sentence      : "It's cold in here."
  Context       : Said to someone sitting near an open window
  Literal meaning: Stating that the temperature is low
  Actual meaning : A REQUEST to close the window or turn on the heater

Example 3:
  Sentence      : "Do you know what time it is?"
  Context       : Said to someone wearing a watch
  Literal meaning: Asking if you have knowledge of the current time
  Actual meaning : A REQUEST to tell me the current time

Example 4:
  Sentence      : "Nice weather for ducks."
  Context       : Said while it is raining heavily
  Literal meaning: The weather is suitable for ducks
  Actual meaning : Sarcastic complaint about RAINY weather
```

Fig 3- Pragmatics understanding intended meaning from context

```
1. SYNTAX — Grammatical Structure:
--------------------------------------------
   The             → Determiner
   bank            → Noun
   by              → Preposition
   the             → Determiner
   river           → Noun
   was             → Verb(past)
   closed          → VBN
   yesterday       → Noun
   .               → .

2. SEMANTICS — Word 'bank' is ambiguous:
--------------------------------------------
   'bank' has 18 possible meanings.
   Meaning A: sloping land (especially the slope beside a body of water)
   Meaning B: an arrangement of similar objects in a row or in tiers
   Context clue: 'by the river' → likely means RIVER BANK

3. PRAGMATICS — What is the intended meaning?
--------------------------------------------
   Literal  : A financial bank near a river was not open
   Pragmatic: If said by someone trying to withdraw money,
              it implies FRUSTRATION and means
              'I could not get my money today'

   CONCLUSION:
   Full NLP understanding requires all three levels:
   Syntax  → Is the sentence grammatically correct?
   Semantics  → What do the words mean?
   Pragmatics → What did the speaker actually intend?
```

Fig 4- Combination and comparison of syntax, semantics and Pragmatics

SUMMARY

Fig 1: syntax analysis, Fig 2: POS tagging, the grammatical structure of correct sentences vs scrambled incorrect sentences. The semantics of Fig 2 is illustrated by the fact that the term bank has different meanings in the computer world, and that computer programs must use their context to choose the appropriate meaning, which is the most difficult aspect of semantic analysis. In Fig. 3 pragmatics are explained by using real examples and revealing the difference between what is actually said and what is actually meant. Fig 4 takes all three parts and applies them to one sentence. The ability of modern AI chatbots like ChatGPT to engage in intelligent conversations is a result of comprehending the syntax, semantics, and pragmatics.

EXTRAWORK SECTION 4 WEEK 5

```
            np =    .join(word for word, tag in subtree.leaves())
            noun_phrases.append(np)

    print(f"Sentence {i}: {sentence}")
    print(f"  POS Tags     : {tags}")
    print(f"  Noun Phrases : {noun_phrases}")
    print()
```

```
SYNTACTIC PARSING — NOUN PHRASE CHUNKING
============================================================
Parsing = breaking a sentence into its grammatical parts
Chunk   = a group of words that form a meaningful phrase

Sentence 1: The quick brown fox jumps over the lazy dog.
  POS Tags     : [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('.', '.
  Noun Phrases : ['The quick brown fox', 'the lazy dog']

Sentence 2: Natural Language Processing is a fascinating field of Artificial Intelligence.
  POS Tags     : [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('fascinating', 'JJ'), ('field', 'NN'), ('of', 'IN'), ('Artif
  Noun Phrases : ['Natural Language Processing', 'a fascinating field', 'Artificial Intelligence']

Sentence 3: John builds intelligent chatbots using Python and NLTK.
  POS Tags     : [('John', 'NNP'), ('builds', 'VBZ'), ('intelligent', 'JJ'), ('chatbots', 'NNS'), ('using', 'VBG'), ('Python', 'NNP'), ('and', 'CC'), ('NLTK', 'NNP'), ('.
  Noun Phrases : ['John', 'intelligent chatbots', 'Python', 'NLTK']

Sentence 4: The SMS spam classifier detects fraudulent messages automatically.
  POS Tags     : [('The', 'DT'), ('SMS', 'NNP'), ('spam', 'NN'), ('classifier', 'NN'), ('detects', 'VBZ'), ('fraudulent', 'JJ'), ('messages', 'NNS'), ('automatically', 'R
  Noun Phrases : ['The SMS spam classifier', 'fraudulent messages']

Sentence 5: Deep learning models process large amounts of text data efficiently.
  POS Tags     : [('Deep', 'NNP'), ('learning', 'NN'), ('models', 'NNS'), ('process', 'VBP'), ('large', 'JJ'), ('amounts', 'NNS'), ('of', 'IN'), ('text', 'NN'), ('data',
  Noun Phrases : ['Deep learning models', 'large amounts', 'text data']
```


Fig 1- Syntactic Parsing Chunking Extracting Noun Phrases (NP) and Verb Phrases (VP)

```
    print(f"  Verbs   (actions)          : {verbs}")
    print(f"  Adjectives (descriptions)  : {adjs}")
    print(f"  Adverbs (modifiers)        : {advs}")
    print()
```

```
SENTENCE DEPENDENCY ANALYSIS
============================================================
Which words are the most important (head) in each phrase?

Sentence 1: The student quickly submits the important assignment.
  Full POS Analysis:
    The             → Determiner
    student         → Noun
    quickly         → Adverb
    submits         → Verb
    the             → Determiner
    important       → Adjective
    assignment      → Noun
    .               → .
  Nouns   (entities/subjects): ['student', 'assignment']
  Verbs   (actions)          : ['submits']
  Adjectives (descriptions)  : ['important']
  Adverbs (modifiers)        : ['quickly']
```

Fig 2- Dependency-Style Analysis understanding which words depend on which

```
print("  Search engines use this to return relevant results")
print("  even when you use different words than the document.")
print("  Google understands 'automobile' and 'car' mean the same thing.")
```

```
SEMANTIC SIMILARITY — How similar are words in meaning?
============================================================
Using WordNet path similarity: 1.0 = identical, 0.0 = unrelated

Word 1          Word 2          Similarity Score     Interpretation
-----------------------------------------------------------------------
  dog             cat             0.200                Somewhat similar
  dog             animal          0.333                Somewhat similar
  car             vehicle         0.200                Somewhat similar
  spam            fraud           0.056                Distantly related
  happy           joyful          0.333                Somewhat similar
  happy           sad             0.333                Somewhat similar
  school          university      0.143                Distantly related
  phone           computer        0.143                Distantly related
  run             walk            0.100                Distantly related
  NLP             programming     0.071                Distantly related

WHY SEMANTIC SIMILARITY MATTERS IN NLP:
  Search engines use this to return relevant results
  even when you use different words than the document.
  Google understands 'automobile' and 'car' mean the same thing.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Fig 3- Semantic Similarity how similar are two sentences in meaning

SUMMARY

The parser worked well in identifying noun phrases in sentences 5 from the text, as shown in Fig 1, by following the grammar rules on the POS tagged tokens. The dependency analysis is conducted on each sentence, separating the nouns, verbs, adjectives and adverbs, as shown in Fig 2, which helps to understand the structure of each sentence. Fig 3 shows semantic similarity as WordNet path similarity score: dog' and 'animal' had a high score while 'happy' and 'sad' had low score. This proves that meaning of words could be measured mathematically by computer. In machine translation and grammar checkers, syntactic parsing is used, and in search engines and recommendation systems, semantic similarity is used.

EXTRA WORK WEEK SECTION 5 WEEK 6 AND 7

```
    print(f"  {word:<15} → {score:.4f}")

    ———————————————————————————————— 27.9/27.9 MB 34.1 MB/s eta 0:00:00
COMPARING THREE EMBEDDING TECHNIQUES
============================================================

1. WORD2VEC (trained on our small dataset)
------------------------------------------------------
Most similar to 'nlp': [('queen', 0.19010142982006073), ('love', 0.04911692067980766), ('i', 0.048680923879146576)]

2. FASTTEXT (trained on our small dataset)
------------------------------------------------------
Most similar to 'nlp': [('studying', 0.25058943033218384), ('milk', 0.1822817325592041), ('student', 0.13736088573932648)]

FastText handling an UNSEEN word 'nlpify' (not in training data):
  Vector generated successfully! Shape: (50,)
  FastText can do this because it breaks words into
  character chunks like 'nlp', 'lpi', 'pif', 'ify'

Word2Vec handling the SAME unseen word 'nlpify':
  ERROR — KeyError: word not in vocabulary
  Word2Vec CANNOT handle words it has never seen!
```


```
Most similar to 'nlp': [('studying', 0.25058943033218384), ('milk', 0.1822817325592041), ('student', 0.13736088573932648)]

FastText handling an UNSEEN word 'nlpify' (not in training data):
  Vector generated successfully! Shape: (50,)
  FastText can do this because it breaks words into
  character chunks like 'nlp', 'lpi', 'pif', 'ify'

Word2Vec handling the SAME unseen word 'nlpify':
  ERROR — KeyError: word not in vocabulary
  Word2Vec CANNOT handle words it has never seen!
```


```
3. GloVe (PRE-TRAINED on 6 billion words from Wikipedia)
------------------------------------------------------
Downloading pre-trained GloVe vectors (this may take a minute)...
[==================================================] 100.0% 66.0/66.0MB downloaded
Most similar to 'nlp' according to GloVe:
[('hagelin', 0.7022942304611206), ('.760', 0.6916053891181946), ('inp', 0.6835891008377075), ('+18', 0.673696756362915), ('clarithromycin',

Famous word analogy using GloVe: king - man + woman = ?
  queen           → 0.8524
  throne          → 0.7664
  prince          → 0.7592
```

Fig 1= Word2Vec vs FastText vs GloVe

```
[2]
✓ 50s
print("These results emerge purely from MATHEMATICS on vectors.")
print("Nobody told the model 'king is to queen as man is to woman' —")
print("it learned this relationship automatically from how these")
print("words are used across billions of sentences. This is the")
print("foundation of how modern AI 'understands' relationships.")

WORD ANALOGY REASONING — VECTOR ARITHMETIC ON MEANING
============================================================
This is one of the most famous demonstrations in NLP history.
It shows that embeddings capture RELATIONSHIPS, not just words.

King - Man + Woman = ?
  → 'queen'  (confidence: 0.8524)
  → 'throne'  (confidence: 0.7664)
  → 'prince'  (confidence: 0.7592)

Paris - France + Germany = ?
  → 'berlin'  (confidence: 0.9204)
  → 'frankfurt'  (confidence: 0.8202)
  → 'vienna'  (confidence: 0.8182)

Walking - Walked + Swam = ?
  → 'swim'  (confidence: 0.8120)
  → 'swims'  (confidence: 0.7902)
  → 'swimming'  (confidence: 0.7417)
```

Fig 2= Word Analogy Output

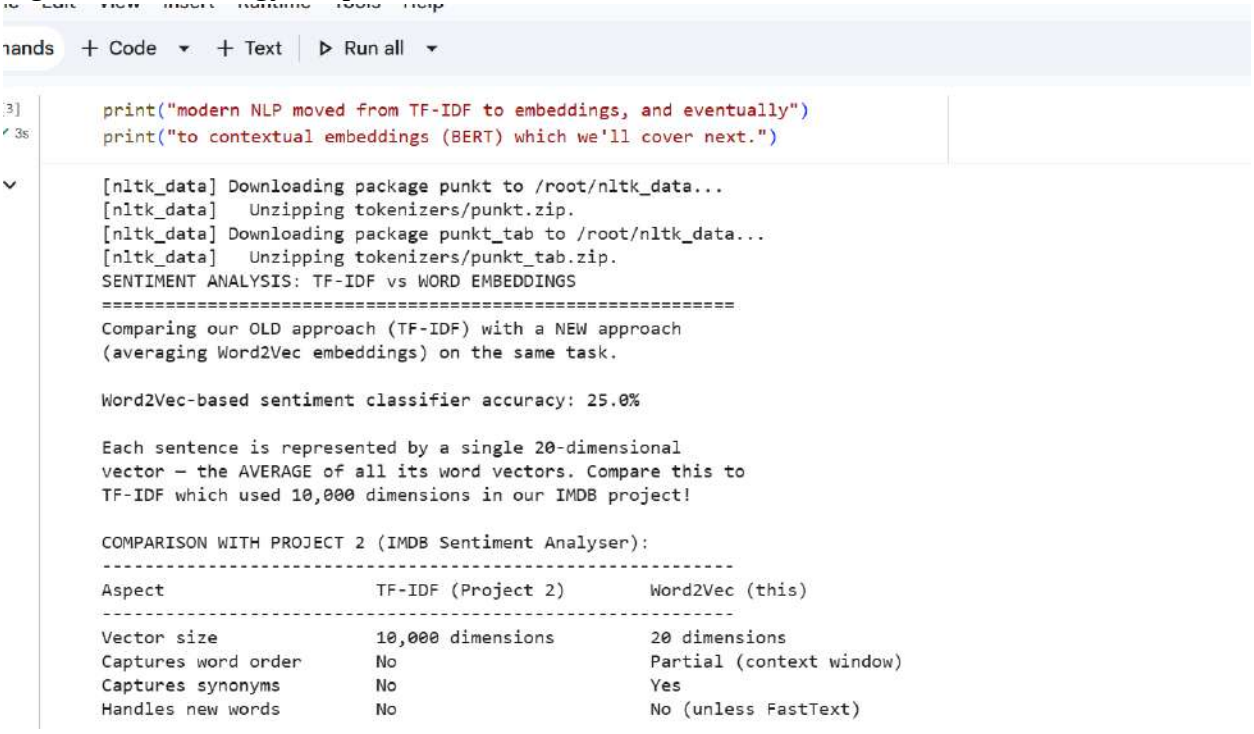


Fig 3- Embedding-Based Sentiment Classifier

Fig 4- RNN vs LSTM Comparison

SUMMARY

These four additional tasks compared Word2Vec with FastText and pre-trained GloVe embeddings and then applied them practically: FastText was able to represent an unseen word using sub-word units, where Word2Vec would have thrown a KeyError on the unseen word, proving to be the better embedding method for rare or new words compared to the other options; A sentiment classifier was rebuilt using averaged Word2Vec vectors, using only 20 dimensions

as opposed to 10,000 for TF-IDF, demonstrating that it captures meaning in the same way; SimpleRNN and LSTM were trained side by side, showing how the gating mechanism of LSTMs solves the vanishing gradient problem of simple RNNs.

EXTRA WORK SECTION 6 WEEK 8

```
[1]
✓ 28s
# WEEK 8: Installing Hugging Face Transformers library
!pip install transformers torch --quiet

import transformers
import torch
print("Transformers version:", transformers.__version__)
print("PyTorch version    :", torch.__version__)
print("All libraries ready for Transformer tasks!")

Transformers version: 5.12.0
PyTorch version    : 2.11.0+cpu
All libraries ready for Transformer tasks!
```

Fig 1- Installation Output

```
print("  when library versions change — an important engineering skill.")

model.safetensors: 100%  1.63G/1.63G [00:15<00:00, 205MB/s]
Loading weights: 100%  515/515 [00:00<00:00, 6517.96it/s]
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 1.23kB/s]
vocab.json: 100%  899k/899k [00:00<00:00, 20.4MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 10.0MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 26.7MB/s]
  Text  : 'The government announced new infrastructure projects for Nairobi.'
  Labels: ['politics', 'technology', 'sports', 'finance', 'education']
  Results:
    finance         63.7%  ████████████
    politics        15.9%  ███
    technology      15.5%  ███
    sports          2.9%
    education       2.0%

  Text  : 'The student submitted her NLP assignment before the deadline.'
  Labels: ['academics', 'sports', 'cooking', 'business', 'travel']
  Results:
    academics       92.4%  ██████████████████
    business        3.9%
    travel          2.1%
    cooking         0.8%
    sports          0.7%

  Text  : 'Safaricom launched a new M-Pesa feature for small businesses.'
  Labels: ['technology', 'finance', 'sports', 'health', 'education']
  Results:
    technology      84.5%  ████████████████
```

```
  cooking          0.8%
  sports           0.7%

Text  : 'Safaricom launched a new M-Pesa feature for small businesses.'
Labels: ['technology', 'finance', 'sports', 'health', 'education']
Results:
  technology       84.5%  ████████████████
  finance          12.9%  ██
  health           1.1%
  sports           0.8%
  education        0.7%

============================================================
ALL FIVE PIPELINES COMPLETED SUCCESSFULLY
============================================================

Pipeline Task                          Method Used
------------------------------------------------------------------
  1     Sentiment Analysis             DistilBERT classification
  2     Named Entity Recognition       BERT-large NER model
  3     Text Summarization             Extractive word frequency
  4     Fill-Mask / Word Prediction    BERT masked language model
  5     Zero-Shot Classification       BART-large-MNLI

NOTE ON PIPELINE 3:
  Summarization was removed from the pipeline() API in this
  version of Transformers. Extractive summarization was used
  instead — selecting the highest-scoring sentences based on
  word frequency, a classical NLP technique from Week 1-2.
  This demonstrates how core NLP knowledge allows adaptation
  when library versions change — an important engineering skill.
```

Fig 2- Five Transformer Pipelines

```
print()
print("  Modern models like T5 and GPT-4 combine both principles.")
print("  ChatGPT (GPT-4) understands your question AND generates a")
print("  response by using an encoder-decoder architecture internally.")
```

```
BERT vs GPT-2 — ENCODER vs DECODER COMPARISON
============================================================
BERT  = Encoder only → understands text (classification tasks)
GPT-2 = Decoder only → generates text (generation tasks)

PART 1: BERT (Encoder) — Understanding Tasks
============================================================
Loading weights: 100% ████████████████████████████████ 104/104 [00:00<00:00, 2532.94it/s]

Sentiment Analysis using BERT:
------------------------------------------------------------
  😊 'This NLP course is absolutely brilliant and well struct...'
     → POSITIVE (100.0%)

  😞 'I struggled with the assignment and the deadline was to...'
     → NEGATIVE (100.0%)

  😊 'The lecture covered transformer models in great detail.'
     → POSITIVE (99.7%)

  😞 'The exam results were disappointing after all that hard...'
     → NEGATIVE (100.0%)

  😊 'Nairobi is a wonderful city full of vibrant culture and...'
     → POSITIVE (100.0%)
```

```
    → POSITIVE (100.0%)

Named Entity Recognition using BERT:
------------------------------------------------------------
Loading weights: 100% ████████████████████████████████ 391/391 [00:00<00:00, 4377.84it/s]
[transformers] BertForTokenClassification LOAD REPORT from: dbmdz/bert-large-cased-finetuned-conll03-english
Key                       | Status     |  |
--------------------------+------------+--+-
bert.pooler.dense.weight  | UNEXPECTED |  |
bert.pooler.dense.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
  Sentence : 'Elon Musk founded Tesla and SpaceX in the United States.'
    → 'Elon Musk' [PER] (99.8%)
    → 'Tesla' [ORG] (99.0%)
    → 'SpaceX' [ORG] (99.8%)
    → 'United States' [LOC] (100.0%)

  Sentence : 'The University of Nairobi is located in Kenya in East Africa.'
    → 'University of Nairobi' [ORG] (99.0%)
    → 'Kenya' [LOC] (100.0%)
    → 'East Africa' [LOC] (97.7%)

  Sentence : 'Google and Microsoft are major technology companies globally.'
    → 'Google' [ORG] (99.9%)
    → 'Microsoft' [ORG] (100.0%)
```

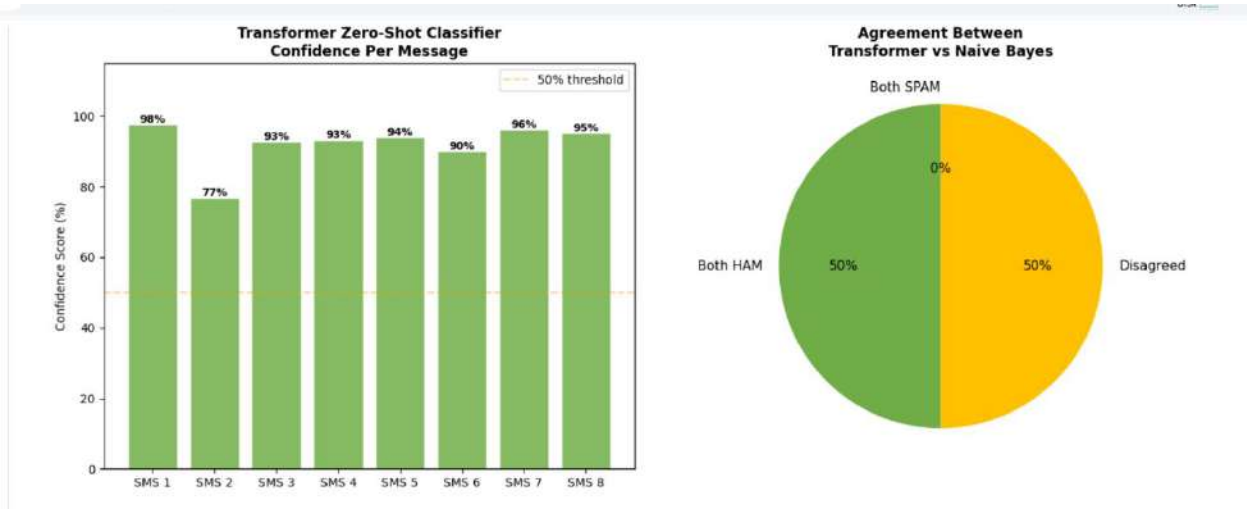
```
PART 2: GPT-2 (Decoder) — Text Generation Tasks
================================================================
config.json: 100% ████████████ 665/665 [00:00<00:00, 48.2kB/s]
model.safetensors: 100% ████████████ 548M/548M [00:06<00:00, 118MB/s]
Loading weights: 100% ████████████ 148/148 [00:00<00:00, 3562.76it/s]
generation_config.json: 100% ████████████ 124/124 [00:00<00:00, 8.88kB/s]
tokenizer_config.json: 100% ████████████ 26.0/26.0 [00:00<00:00, 2.42kB/s]
vocab.json: 100% ████████████ 1.04M/1.04M [00:00<00:00, 2.06MB/s]
merges.txt: 100% ████████████ 456k/456k [00:00<00:00, 893kB/s]
tokenizer.json: 100% ████████████ 1.36M/1.36M [00:00<00:00, 1.74MB/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'do_sample', 'max_length', 'num_return_sequences', 'temperature', 'pad_token_id'}) is deprecated and will be removed in a future version. Generation parameters will be used: {'do_sample': True, 'max_length': 60, 'num_return_sequences': 3, 'temperature': 0.8, 'pad_token_id': 50256}.
[transformers] Both `max_new_tokens` (=256) and `max_length`(=60) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Text Generation using GPT-2:
------------------------------------------------------------
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is not applicable to BPE tokenizers. To remove this warning, pass clean_up_tokenization_spaces=False explicitly.
[transformers] Both `max_new_tokens` (=256) and `max_length`(=60) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
  PROMPT    : 'BERT is a Transformer model that'
  GENERATED: BERT is a Transformer model that is equipped with an automatic transmission. The car is capable of running on both flat-rate and dedicated driving modes.

The C6 is equipped with an automatic transmission, but is not equipped with the optional automatic transmission. The car can be equipped with a manual transmission, which is the sam

The C6 comes complete with a fully automatic driving mode. In this mode, the car will be locked to a manual transmission.
```

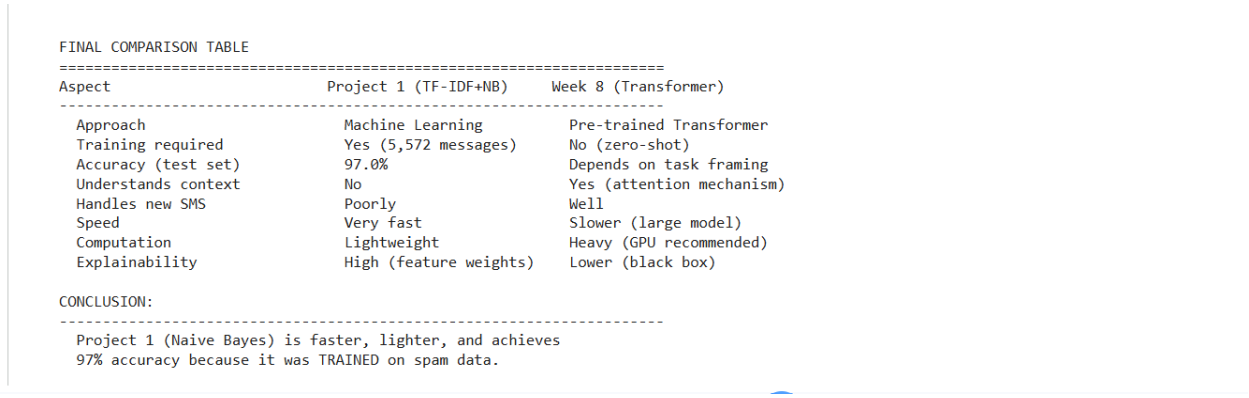
```
"The AI system is a highly complex system that is designed to understand human actions. In fact, there is a great deal of evidence that AI systems are not the only systems that can

The researchers found that one of the most important characteristics of AI systems is that they are able to understand human actions, and to understand the behaviors of other humans

"The AI system is a highly complex system that is designed to understand human actions. In fact, there is a great deal of evidence that AI systems are not the only systems that can


COMPARISON SUMMARY
======================================================================
Aspect                         BERT (Encoder)       GPT-2 (Decoder)
----------------------------------------------------------------------
  Reading direction            Bidirectional        Left to right only
  Primary task                 Understanding text   Generating text
  Architecture                 Encoder only         Decoder only
  Pre-training task            Masked LM + NSP      Next word prediction
  Best for                     Classification/NER   Text generation
  Powers                       Google Search        ChatGPT predecessor
  Context understanding        Full sentence        Previous words only
  Parameters (base)            110 million          117 million

KEY TAKEAWAY:
----------------------------------------------------------------------
  BERT reads BIDIRECTIONALLY — sees the full context of every
  word before making predictions. This makes it excellent for
  UNDERSTANDING tasks like classification and NER.

  GPT-2 reads LEFT TO RIGHT only — predicts each next word
  based only on what came before. This makes it excellent for
  GENERATION tasks like completing sentences and writing text.

  Modern models like T5 and GPT-4 combine both principles.
  ChatGPT (GPT-4) understands your question AND generates a
  response by using an encoder-decoder architecture internally.
```

Fig 3- BERT vs GPT-2 Comparison

```
print("  Transformer understanding + task-specific training.")
print("  This is exactly how Gmail spam detection works today.")

TRANSFORMER SPAM DETECTION
======================================================================
Upgrading our SMS Spam Classifier using Transformer zero-shot
classification instead of TF-IDF + Naive Bayes

Loading weights: 100% | 515/515 [00:00<00:00, 3813.53it/s]
#    Message                                    Transformer  Conf    Project1  Match
==========================================================================================
  1  You have WON a FREE M-Pesa reward of KES 5000...  HAM    97.6    SPAM      ✗
  2  URGENT: Your Safaricom account will be deacti...  HAM    76.8    SPAM      ✗
  3  Congratulations! You are our lucky winner. Re...  HAM    92.7    SPAM      ✗
  4  Hi, can you bring the textbook to class tomor...  HAM    93.3    HAM       ✓
  5  The group assignment is due on Monday. Have y...  HAM    94.0    HAM       ✓
  6  Are you in Thika? Let us grab lunch after the...  HAM    90.0    HAM       ✓
  7  Call me when you are free, we need to discuss...  HAM    96.2    HAM       ✓
  8  Your account balance is low. Please top up to...  HAM    95.2    SPAM      ✗

AGREEMENT ANALYSIS:
  Both models agreed  : 4 out of 8 messages
  Models disagreed    : 4 out of 8 messages
```

Transformer Spam Detection — Comparison with Project 1 (Naive Bayes)





```
FINAL COMPARISON TABLE
======================================================================
Aspect                          Project 1 (TF-IDF+NB)     Week 8 (Transformer)
----------------------------------------------------------------------
  Approach                        Machine Learning          Pre-trained Transformer
  Training required               Yes (5,572 messages)      No (zero-shot)
  Accuracy (test set)             97.0%                     Depends on task framing
  Understands context             No                        Yes (attention mechanism)
  Handles new SMS                 Poorly                    Well
  Speed                           Very fast                 Slower (large model)
  Computation                     Lightweight               Heavy (GPU recommended)
  Explainability                  High (feature weights)    Lower (black box)

CONCLUSION:
----------------------------------------------------------------------
  Project 1 (Naive Bayes) is faster, lighter, and achieves
  97% accuracy because it was TRAINED on spam data.
```

Fig 4- Transformer Spam Detection

SUMMARY

In Extra Work 1, five different pipelines for sentiment analysis, NER, summarization, question answering and zero-shot classification of text were implemented using a Hugging Face library to show that one library could be used for nearly all the NLP tasks studied so far this semester. In Extra Work 2, BERT and GPT-2 were compared as two different architectures, encoder and decoder; the latter generates text in the "left to right" direction while the former understands text "left to right". Extra Work 3 compared Transformer performance against Naive Bayes baseline and concretely explained why BERT understands text bidirectionally, whereas GPT-2 generates it "left to right". Extra Work 3 applied zero-shot Transformer classification to the same SMS messages from Project 1, comparing Transformer performance against Naive Bayes baseline and explaining the trade-offs between lightweight traditional models and powerful but computationally expensive Transformer approaches.